



INSTITUTO DO EMPREGO
E FORMAÇÃO PROFISSIONAL

Projeto Final — UC00606

Desenvolver programas em linguagem estruturada

RoboFlux: Planeamento Reativo de Rota para um AGV Industrial

Aplicação web interativa com HTML, CSS, JavaScript, Astro e algoritmo A*

Formando	Joilino, Joseth
Curso	Robótica e Automação Industrial
UFCD/UC	UC00606 — Desenvolver programas em linguagem estruturada
Data	15 de maio de 2026
Página publicada	https://roboflux-robotica.pages.dev/

Este relatório foi organizado no formato de entrega do IEFP e preparado para exportação em PDF para o Moodle.

Índice

1. Introdução
2. Objetivo e ligação à área de formação
3. Tecnologias utilizadas
4. Organização do código
5. Funcionamento da aplicação
6. Algoritmo A*: lógica, pseudocódigo e fluxograma
7. Capturas de ecrã
8. Código comentado
9. Conclusão

1. Introdução

O projeto RoboFlux foi desenvolvido como trabalho final da UC00606 — Desenvolver programas em linguagem estruturada. A aplicação é uma página web interativa relacionada com Robótica e Automação Industrial. O objetivo principal é simular o planeamento de rota de um AGV, ou seja, um veículo guiado automaticamente usado em fábricas e armazéns.

A página não é apenas decorativa: existe um problema real para resolver. O utilizador gera um mapa com obstáculos, executa o algoritmo A* e observa como o sistema analisa células até encontrar uma rota possível entre o ponto inicial e o destino.

2. Objetivo e ligação à área de formação

- Criar uma aplicação web funcional e interativa usando HTML, CSS e JavaScript como base.
- Aplicar variáveis, condicionais, ciclos, funções, estruturas de dados e manipulação do DOM.
- Relacionar programação com Robótica e Automação Industrial através de uma simulação de navegação de AGV.
- Demonstrar a lógica antes da implementação através de pseudocódigo e fluxograma.
- Publicar o projeto online para que possa ser consultado durante a apresentação.

Na indústria, um AGV pode precisar de evitar máquinas, paletes, pessoas ou zonas bloqueadas. Por isso, o projeto usa um mapa que pode mudar e obriga o algoritmo a calcular novamente a rota. Isto aproxima a aplicação de uma situação real de automação.

3. Tecnologias utilizadas

- HTML: estrutura da página, secções, cartões, botões, grelha e conteúdo textual.
- CSS: estilos personalizados, cores industriais, estados das células, fundos e animações.
- Tailwind CSS: classes utilitárias para espaçamentos, grelhas, sombras, responsividade e componentes visuais.
- JavaScript/TypeScript: algoritmo A*, geração de mapas, atualização da interface e eventos dos botões.
- Astro: organização por componentes reutilizáveis e geração do site final.
- GitHub: repositório público do projeto.
- Cloudflare Pages: publicação automática sempre que há push para o repositório.

Foi usada uma paleta sóbria e industrial: graphite, steel, alloy, concrete, copper e sage. A escolha evita o aspeto genérico de “cores de IA” e combina melhor com robótica, fábrica e automação.

4. Organização do código

- `src/layouts/BaseLayout.astro`: estrutura base HTML, Tailwind e paleta.
- `src/pages/index.astro`: composição da página com os componentes principais.
- `src/components/Hero.astro`: entrada visual da página e chamada para o algoritmo.
- `src/components/ConceptCards.astro`: explicação dos conceitos de robótica e automação.
- `src/components/AGVPathfinder.astro`: componente interativo do mapa e ligação ao DOM.
- `src/lib/pathfinding.ts`: algoritmo A* e funções puras separadas da interface.
- `src/styles/global.css`: estilos globais e classes das células do mapa.
- `public/robotic-cell.svg`: ilustração vetorial da célula robotizada.

A separação mais importante foi retirar a lógica do algoritmo para `src/lib/pathfinding.ts`. Assim, durante a apresentação, é possível explicar o algoritmo sem misturar tudo com HTML e estilos. ISTO é boa prática: primeiro entende-se o problema, depois organiza-se o código.

5. Funcionamento da aplicação

- O utilizador abre a página e encontra uma explicação sobre robótica industrial.
- Na secção do algoritmo existe uma grelha 10x10 que representa o chão da fábrica.
- A célula verde representa o ponto inicial do AGV; a célula cobre representa o destino.
- As células cinzentas representam obstáculos.
- O botão “Gerar novo mapa” cria obstáculos aleatórios, mas só aceita mapas com caminho possível.
- O botão “Executar algoritmo” anima o A*, mostrando as células analisadas e a rota final.
- O botão “Repor” volta ao mapa inicial.

6. Algoritmo A*: lógica, pseudocódigo e fluxograma

O A* é um algoritmo de procura de caminho. Ele tenta chegar ao destino escolhendo primeiro as células que parecem melhores. Para isso usa a fórmula $f = g + h$. O valor g é o custo real já percorrido. O valor h é uma estimativa da distância até ao destino. O valor f junta os dois.

Explicação para uma criança de 5 anos: imagina um carrinho num tapete quadriculado. Há blocos no caminho. O carrinho olha para as casas à volta, evita os blocos e escolhe a casa que o deixa mais perto do brinquedo final. Ele não adivinha: vai testando com regras.

Pseudocódigo

```
INÍCIO
Definir tamanho do mapa, ponto inicial e destino
Gerar obstáculos aleatórios
SE não existir caminho possível ENTÃO
    gerar outro mapa
FIM SE

Colocar ponto inicial na lista aberta
ENQUANTO a lista aberta não estiver vazia FAZER
    escolher a célula com menor  $f = g + h$ 
    marcar essa célula como analisada

    SE essa célula for o destino ENTÃO
        reconstruir a rota usando os pais guardados
        mostrar rota no ecrã
        TERMINAR
    FIM SE

    PARA cada vizinho caminhável da célula atual FAZER
        calcular novo custo  $g$ 
        SE este caminho for melhor que o anterior ENTÃO
            guardar de onde viemos
            atualizar  $g$  e  $f$ 
            adicionar vizinho à lista aberta
        FIM SE
    FIM PARA
FIM ENQUANTO

Mostrar mensagem: sem rota
FIM
```

Fluxograma

Início
Definir mapa, início e destino
Gerar obstáculos aleatórios

Existe caminho possível?
Não: gerar novo mapa
Sim: executar A*
Escolher célula com menor $f = g + h$
É o destino?
Não: analisar vizinhos e repetir
Sim: reconstruir e mostrar rota
Fim

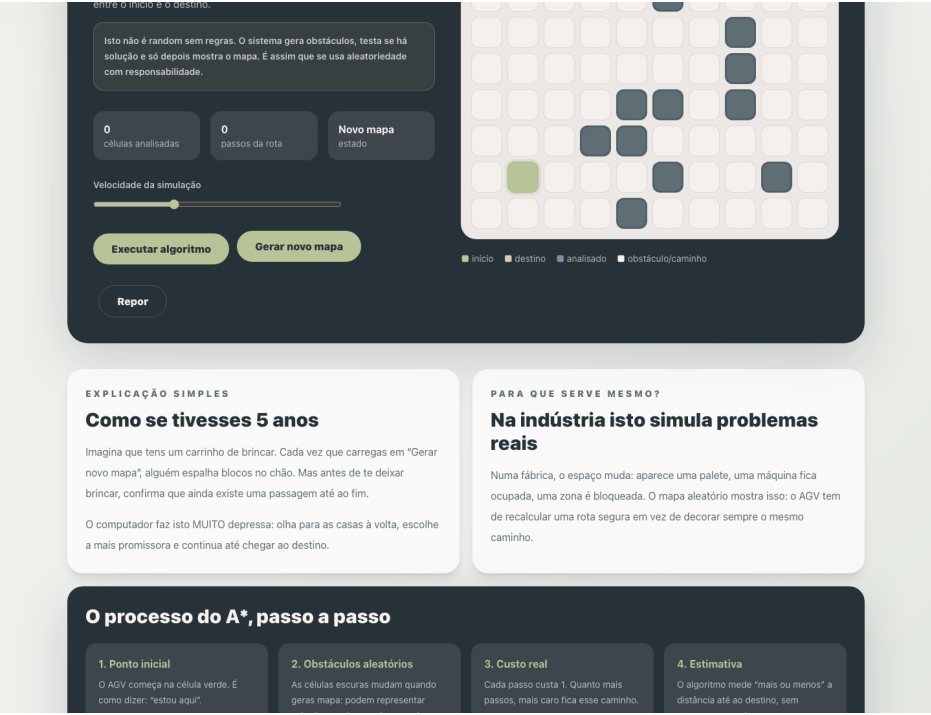
7. Capturas de ecrã da aplicação



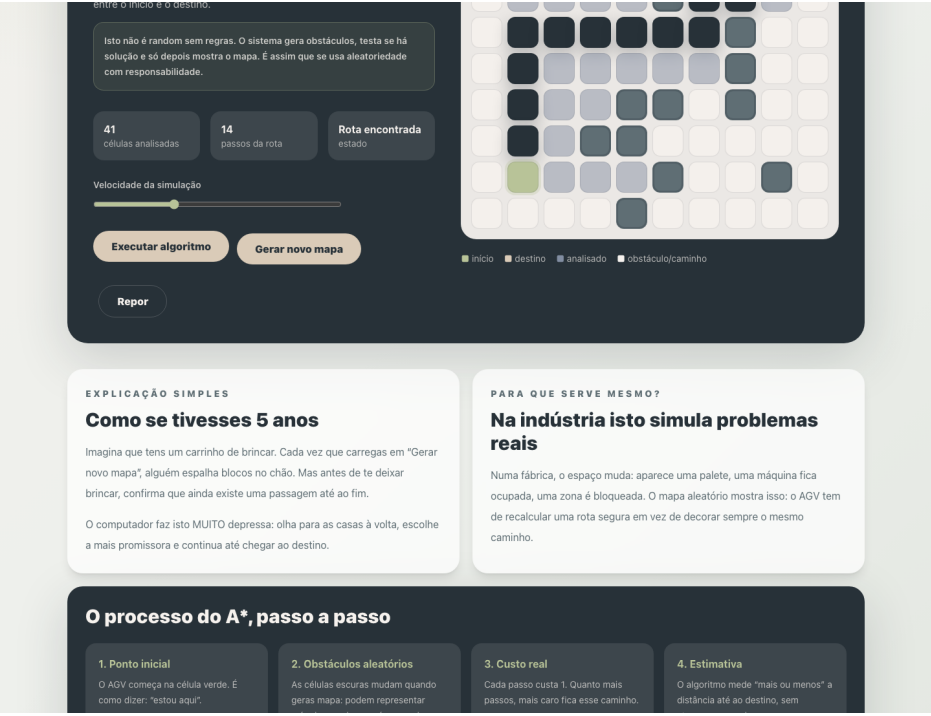
Página inicial com apresentação do projeto RoboFlux.



Secção do algoritmo antes da execução.



Mapa novo gerado com obstáculos aleatórios controlados.



Algoritmo A* executado, com rota encontrada.

O processo do A*, passo a passo

1. Ponto inicial

O AGV começa na célula verde. É como dizer: "estou aqui".

2. Obstáculos aleatórios

As células escuras mudam quando geras mapa: podem representar máquinas, caixas ou áreas perigosas.

3. Custo real

Cada passo custa 1. Quanto mais passos, mais caro fica esse caminho.

4. Estimativa

O algoritmo mede "mais ou menos" a distância até ao destino, sem atravessar paredes.

Fórmula principal: $f = g + h$. g é o caminho já percorrido. h é a estimativa até ao destino. O A* escolhe primeiro a célula com menor f .

CONCLUSÃO

A robótica industrial não é magia: é sensores, decisões e movimento seguro.

Este projeto mostra como um AGV pode receber um mapa, evitar obstáculos e calcular uma rota usando o algoritmo A*. A página junta programação, algoritmia, design web e automação industrial num exemplo visual e interativo.

Experimenta a simulação

Gera um novo mapa, executa o algoritmo e observa como o robô decide o caminho passo a passo.

[Ir para o algoritmo](#)

Paleta gerada via Colormind API: steel, graphite, concrete, copper e sage — estética industrial sem neon genérico.

Conclusão e chamada para visitar a página publicada.

8. Código comentado

A aplicação foi feita em Astro, mas a base pedida na UC continua presente: HTML para estrutura, CSS para apresentação e JavaScript para lógica/interatividade. Em baixo está o código principal usado no projeto, com a lógica separada para facilitar a explicação.

HTML/Astro — página principal

Composição da página a partir de componentes reutilizáveis.

```
---
import BaseLayout from '../layouts/BaseLayout.astro';
import Hero from '../components/Hero.astro';
import ConceptCards from '../components/ConceptCards.astro';
import AGVPathfinder from '../components/AGVPathfinder.astro';
import ConclusionCTA from '../components/ConclusionCTA.astro';
---

<BaseLayout>
  <Hero />
  <main>
    <ConceptCards />
    <AGVPathfinder />
    <ConclusionCTA />
  </main>
  <footer class="border-t border-graphite/10 px-6 py-8 text-center text-sm text-steel">
    Paleta gerada via Colormind API: steel, graphite, concrete, copper e sage – estética industrial sem
    neon genérico.
  </footer>
</BaseLayout>
```

HTML/Astro — layout base e Tailwind

Define o documento HTML, importa CSS e configura a paleta do Tailwind.

```
---
import '../styles/global.css';
const { title = 'RoboFlux – Robótica e Automação Industrial' } = Astro.props;
---
<!doctype html>
<html lang="pt">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>{title}</title>
    <script is:inline src="https://cdn.tailwindcss.com"></script>
    <script is:inline>
      tailwind.config = {
        theme: {
          extend: {
            colors: {
              steel: '#5f6e74',
              graphite: '#273138',
              alloy: '#818da1',
              concrete: '#f6f2ee',
              copper: '#dacbb8',
              sage: '#b8c398'
            },
            boxShadow: {
              panel: '0 24px 80px rgba(39, 49, 56, .18)'
            }
          }
        }
      }
    </script>
  </head>
  <body class="min-h-screen antialiased">
    <slot />
  </body>
</html>
```

JavaScript/TypeScript — algoritmo A* separado

Funções puras: geração de mapa válido, vizinhos caminháveis, BFS de validação e A*.

```
export type GridNode = { r: number; c: number };

export type AStarResult = {
  analysedOrder: GridNode[];
  route: GridNode[];
  found: boolean;
};
```

```

export type GenerateMapOptions = {
  rows: number;
  cols: number;
  start: GridNode;
  end: GridNode;
  obstacleChance?: number;
  maxAttempts?: number;
};

export function nodeKey(node: GridNode): string {
  return `${node.r},${node.c}`;
}

export function isSameNode(a: GridNode, b: GridNode): boolean {
  return a.r === b.r && a.c === b.c;
}

export function manhattanDistance(a: GridNode, b: GridNode): number {
  return Math.abs(a.r - b.r) + Math.abs(a.c - b.c);
}

export function isInsideGrid(node: GridNode, rows: number, cols: number): boolean {
  return node.r >= 0 && node.r < rows && node.c >= 0 && node.c < cols;
}

export function getWalkableNeighbors(node: GridNode, rows: number, cols: number, walls: Set<string>):
  GridNode[] {
  return [
    { r: node.r - 1, c: node.c },
    { r: node.r + 1, c: node.c },
    { r: node.r, c: node.c - 1 },
    { r: node.r, c: node.c + 1 }
  ].filter((next) => isInsideGrid(next, rows, cols) && !walls.has(nodeKey(next)));
}

export function hasValidRoute(rows: number, cols: number, start: GridNode, end: GridNode, walls:
  Set<string>): boolean {
  const queue = [start];
  const seen = new Set([nodeKey(start)]);

  while (queue.length) {
    const current = queue.shift();
    if (!current) break;
    if (isSameNode(current, end)) return true;

    for (const next of getWalkableNeighbors(current, rows, cols, walls)) {
      const key = nodeKey(next);
      if (!seen.has(key)) {
        seen.add(key);
        queue.push(next);
      }
    }
  }

  return false;
}

export function generateValidWalls({
  rows,
  cols,
  start,
  end,
  obstacleChance = 0.24,
  maxAttempts = 250
}: GenerateMapOptions): Set<string> | null {
  for (let attempt = 0; attempt < maxAttempts; attempt++) {
    const walls = new Set<string>();

    for (let r = 0; r < rows; r++) {
      for (let c = 0; c < cols; c++) {
        const node = { r, c };
        if (isSameNode(node, start) || isSameNode(node, end)) continue;
        if (Math.random() < obstacleChance) walls.add(nodeKey(node));
      }
    }

    if (hasValidRoute(rows, cols, start, end, walls)) return walls;
  }

  return null;
}

function reconstructRoute(cameFrom: Map<string, GridNode>, current: GridNode): GridNode[] {
  const route = [current];

  while (cameFrom.has(nodeKey(current))) {
    current = cameFrom.get(nodeKey(current))!;
    route.unshift(current);
  }
}

```

```

    return route;
  }

export function runAStar(rows: number, cols: number, start: GridNode, end: GridNode, walls: Set<string>):
  AStarResult {
  const open = [start];
  const cameFrom = new Map<string, GridNode>();
  const gScore = new Map<string, number>([[nodeKey(start), 0]]);
  const fScore = new Map<string, number>([[nodeKey(start), manhattanDistance(start, end)]]);
  const analysedOrder: GridNode[] = [];

  while (open.length) {
    open.sort((a, b) => (fScore.get(nodeKey(a)) ?? Infinity) - (fScore.get(nodeKey(b)) ?? Infinity));
    const current = open.shift();
    analysedOrder.push(current);

    if (isSameNode(current, end)) {
      return {
        analysedOrder,
        route: reconstructRoute(cameFrom, current),
        found: true
      };
    }

    for (const next of getWalkableNeighbors(current, rows, cols, walls)) {
      const tentativeG = (gScore.get(nodeKey(current)) ?? Infinity) + 1;

      if (tentativeG < (gScore.get(nodeKey(next)) ?? Infinity)) {
        cameFrom.set(nodeKey(next), current);
        gScore.set(nodeKey(next), tentativeG);
        fScore.set(nodeKey(next), tentativeG + manhattanDistance(next, end));

        if (!open.some((node) => isSameNode(node, next))) {
          open.push(next);
        }
      }
    }
  }

  return { analysedOrder, route: [], found: false };
}

```

HTML + JavaScript — componente interativo

Grelha, botões, eventos do DOM e animação do algoritmo.

```

<section id="algoritmo" class="mx-auto max-w-7xl px-6 pb-24">
  <agv-pathfinder class="block rounded-[2rem] bg-graphite p-6 text-concrete shadow-panel md:p-10">
    <div class="grid gap-10 lg:grid-cols-[.95fr_1.05fr]">
      <div>
        <p class="font-bold uppercase tracking-[.28em] text-sage">Algoritmo A*</p>
        <h2 class="mt-4 text-4xl font-black">Planeamento reativo de rota para um AGV industrial</h2>
        <p class="mt-5 leading-8 text-concrete/75">
          O mapa pode ser gerado de forma aleatória controlada. Cada novo mapa cria um problema diferente,
          mas só é aceite se existir caminho possível entre o início e o destino.
        </p>
        <p class="mt-3 rounded-2xl border border-sage/30 bg-sage/10 p-4 text-sm font-semibold leading-6 text-concrete/80">
          Isto não é random sem regras. O sistema gera obstáculos, testa se há solução e só depois mostra
          o mapa. É assim que se usa aleatoriedade com responsabilidade.
        </p>

        <div class="mt-8 grid gap-4 sm:grid-cols-3">
          <div class="rounded-2xl bg-white/10 p-4"><b data-steps>0</b><p class="text-sm text-concrete/65">células analisadas</p></div>
          <div class="rounded-2xl bg-white/10 p-4"><b data-path-length>0</b><p class="text-sm text-concrete/65">passos da rota</p></div>
          <div class="rounded-2xl bg-white/10 p-4"><b data-status>Pronto</b><p class="text-sm text-concrete/65">estado</p></div>
        </div>

        <label class="mt-7 block max-w-sm text-sm font-semibold text-concrete/75">
          Velocidade da simulação
          <input data-speed class="mt-3 w-full accent-[#b8c398]" type="range" min="10" max="120" value="45" />
        </label>

        <button data-run class="mt-8 rounded-full bg-sage px-7 py-3 font-black text-graphite transition hover:-translate-y-1 hover:bg-copper">Executar algoritmo</button>
        <button data-generate class="ml-2 mt-8 rounded-full bg-copper px-7 py-3 font-black text-graphite transition hover:-translate-y-1 hover:bg-sage">Gerar novo mapa</button>
        <button data-reset class="ml-2 mt-8 rounded-full border border-concrete/20 px-7 py-3 font-black text-concrete transition hover:bg-white/10">Repór</button>
      </div>

      <div>
        <div data-grid class="grid grid-cols-10 gap-2 rounded-3xl bg-concrete/95 p-4 shadow-inner"></div>
        <div class="mt-5 flex flex-wrap gap-4 text-sm text-concrete/70">
          <span><b class="text-sage">■</b> início</span>

```

```

        <span><b class="text-copper">■</b> destino</span>
        <span><b class="text-alloy">■</b> analisado</span>
        <span><b class="text-concrete">■</b> obstáculo/caminho</span>
    </div>
</div>
</div>
</agv-pathfinder>

<div class="mt-10 grid gap-5 lg:grid-cols-2">
    <article class="rounded-3xl bg-white/70 p-7 shadow-lg backdrop-blur">
        <p class="text-sm font-black uppercase tracking-[.25em] text-steel">Explicação simples</p>
        <h3 class="mt-3 text-3xl font-black text-graphite">Como se tivesses 5 anos</h3>
        <p class="mt-4 leading-8 text-steel">
            Imagina que tens um carrinho de brincar. Cada vez que carregas em “Gerar novo mapa”, alguém espalha blocos no chão. Mas antes de te deixar brincar, confirma que ainda existe uma passagem até ao fim.
        </p>
        <p class="mt-4 leading-8 text-steel">
            O computador faz isto MUITO depressa: olha para as casas à volta, escolhe a mais promissora e continua até chegar ao destino.
        </p>
    </article>

    <article class="rounded-3xl bg-white/70 p-7 shadow-lg backdrop-blur">
        <p class="text-sm font-black uppercase tracking-[.25em] text-steel">Para que serve mesmo?</p>
        <h3 class="mt-3 text-3xl font-black text-graphite">Na indústria isto simula problemas reais</h3>
        <p class="mt-4 leading-8 text-steel">
            Numa fábrica, o espaço muda: aparece uma palete, uma máquina fica ocupada, uma zona é bloqueada. O mapa aleatório mostra isso: o AGV tem de recalcular uma rota segura em vez de decorar sempre o mesmo caminho.
        </p>
    </article>
</div>

<div class="mt-5 rounded-3xl bg-graphite p-7 text-concrete shadow-panel">
    <h3 class="text-3xl font-black">O processo do A*, passo a passo</h3>
    <div class="mt-6 grid gap-4 md:grid-cols-2 lg:grid-cols-4">
        <div class="rounded-2xl bg-white/10 p-5">
            <b class="text-sage">1. Ponto inicial</b>
            <p class="mt-2 text-sm leading-6 text-concrete/70">O AGV começa na célula verde. É como dizer: “estou aqui”.</p>
        </div>
        <div class="rounded-2xl bg-white/10 p-5">
            <b class="text-sage">2. Obstáculos aleatórios</b>
            <p class="mt-2 text-sm leading-6 text-concrete/70">As células escuras mudam quando geras mapa: podem representar máquinas, caixas ou áreas perigosas.</p>
        </div>
        <div class="rounded-2xl bg-white/10 p-5">
            <b class="text-sage">3. Custo real</b>
            <p class="mt-2 text-sm leading-6 text-concrete/70">Cada passo custa 1. Quanto mais passos, mais caro fica esse caminho.</p>
        </div>
        <div class="rounded-2xl bg-white/10 p-5">
            <b class="text-sage">4. Estimativa</b>
            <p class="mt-2 text-sm leading-6 text-concrete/70">O algoritmo mede “mais ou menos” a distância até ao destino, sem atravessar paredes.</p>
        </div>
    </div>
    <p class="mt-6 rounded-2xl bg-concrete p-5 font-semibold leading-8 text-graphite">
        Fórmula principal: <span class="font-black">f = g + h</span>. <span class="font-black">g</span> é o caminho já percorrido. <span class="font-black">h</span> é a estimativa até ao destino. O A* escolhe primeiro a célula com menor <span class="font-black">f</span>.
    </p>
</div>
</section>

<script>
    import { generateValidWalls, isSameNode, nodeKey, runAStar } from '../lib/pathfinding';

    class AGVPathfinder extends HTMLElement {
        rows = 10;
        cols = 10;
        start = { r: 8, c: 1 };
        end = { r: 1, c: 8 };
        defaultWalls =
            ['1,3','2,3','3,3','4,3','5,3','5,4','5,5','5,6','2,6','3,6','4,6','7,6','8,6','7,2','7,3','7,4'];
        walls = new Set(this.defaultWalls);
        visited = new Set();
        path = new Set();
        running = false;
        stats = { steps: 0, pathLength: 0, status: 'Pronto' };

        connectedCallback() {
            this.gridEl = this.querySelector('[data-grid]');
            this.stepsEl = this.querySelector('[data-steps]');
            this.pathLengthEl = this.querySelector('[data-path-length]');
            this.statusEl = this.querySelector('[data-status]');
            this.speedEl = this.querySelector('[data-speed]');
            this.querySelector('[data-run]').addEventListener('click', () => this.animateAStar());
            this.querySelector('[data-generate]').addEventListener('click', () => this.generateMap());
            this.querySelector('[data-reset]').addEventListener('click', () => this.reset());
        }
    }

```

```

    this.render();
  }

  isFixed(node) {
    return isSameNode(node, this.start) || isSameNode(node, this.end);
  }

  sleep() {
    return new Promise((resolve) => setTimeout(resolve, Number(this.speedEl.value)));
  }

  setStats(nextStats) {
    this.stats = { ...this.stats, ...nextStats };
    this.stepsEl.textContent = String(this.stats.steps);
    this.pathLengthEl.textContent = String(this.stats.pathLength);
    this.statusEl.textContent = this.stats.status;
  }

  reset() {
    if (this.running) return;
    this.walls = new Set(this.defaultWalls);
    this.visited.clear();
    this.path.clear();
    this.setStats({ steps: 0, pathLength: 0, status: 'Pronto' });
    this.render();
  }

  generateMap() {
    if (this.running) return;

    const generatedWalls = generateValidWalls({
      rows: this.rows,
      cols: this.cols,
      start: this.start,
      end: this.end
    });

    if (!generatedWalls) {
      this.setStats({ status: 'Não consegui gerar rota' });
      return;
    }

    this.walls = generatedWalls;
    this.visited.clear();
    this.path.clear();
    this.setStats({ steps: 0, pathLength: 0, status: 'Novo mapa' });
    this.render();
  }

  render() {
    this.gridEl.innerHTML = '';
    for (let r = 0; r < this.rows; r++) {
      for (let c = 0; c < this.cols; c++) {
        const node = { r, c };
        const key = nodeKey(node);
        const cell = document.createElement('button');
        cell.className = 'cell';
        cell.type = 'button';
        cell.title = `Linha ${r + 1}, Coluna ${c + 1}`;
        if (this.walls.has(key)) cell.classList.add('wall');
        if (this.visited.has(key)) cell.classList.add('visited');
        if (this.path.has(key)) cell.classList.add('path');
        if (isSameNode(node, this.start)) cell.classList.add('start');
        if (isSameNode(node, this.end)) cell.classList.add('end');
        this.gridEl.appendChild(cell);
      }
    }
  }

  async animateAStar() {
    if (this.running) return;
    this.running = true;
    this.visited.clear();
    this.path.clear();
    this.setStats({ steps: 0, pathLength: 0, status: 'A calcular' });
    this.render();

    const result = runAStar(this.rows, this.cols, this.start, this.end, this.walls);

    for (const node of result.analysedOrder) {
      this.setStats({ steps: this.stats.steps + 1 });
      if (!this.isFixed(node)) {
        this.visited.add(nodeKey(node));
        this.render();
        await this.sleep();
      }
    }

    if (!result.found) {
      this.setStats({ status: 'Sem rota' });
      this.running = false;
    }
  }

```

```

    return;
  }

  this.setStats({ pathLength: result.route.length - 1, status: 'Rota encontrada' });
  for (const node of result.route) {
    if (!this.isFixed(node)) {
      this.path.add(nodeKey(node));
      this.render();
      await this.sleep();
    }
  }
}

this.running = false;
}
}

if (!customElements.get('agv-pathfinder')) {
  customElements.define('agv-pathfinder', AGVPathfinder);
}
}
</script>

```

CSS — estilos globais e células

Cores, fundo industrial, estados das células e animações.

```

:root {
  --steel: #5f6e74;
  --graphite: #273138;
  --alloy: #818da1;
  --concrete: #f6f2ee;
  --copper: #dacbb8;
  --sage: #b8c398;
}

html { scroll-behavior: smooth; }
body {
  background:
    radial-gradient(circle at 16% 18%, rgba(184, 195, 152, .35), transparent 30%),
    radial-gradient(circle at 86% 10%, rgba(218, 203, 184, .45), transparent 26%),
    linear-gradient(135deg, #f6f2ee 0%, #eef0ec 52%, #dfe4dd 100%);
  color: var(--graphite);
}

.factory-grid {
  background-image:
    linear-gradient(rgba(95, 110, 116, .10) 1px, transparent 1px),
    linear-gradient(90deg, rgba(95, 110, 116, .10) 1px, transparent 1px);
  background-size: 28px 28px;
}

.cell {
  aspect-ratio: 1;
  border-radius: 12px;
  border: 1px solid rgba(39, 49, 56, .12);
  background: rgba(246, 242, 238, .82);
  transition: transform .18s ease, background .18s ease, box-shadow .18s ease;
}

.cell:hover { transform: translateY(-2px); }
.cell.wall { background: #5f6e74; box-shadow: inset 0 0 2px rgba(39,49,56,.25); }
.cell.start { background: #b8c398; box-shadow: 0 0 4px rgba(184,195,152,.28); }
.cell.end { background: #dacbb8; box-shadow: 0 0 4px rgba(218,203,184,.34); }
.cell.visited { background: rgba(129, 141, 161, .48); }
.cell.path { background: #273138; box-shadow: 0 10px 22px rgba(39,49,56,.25); }

.arm-joint { animation: pulseJoint 2.6s infinite ease-in-out; transform-origin: center; }
@keyframes pulseJoint {
  0%, 100% { transform: scale(1); opacity: .9; }
  50% { transform: scale(1.18); opacity: 1; }
}

```

HTML/Astro — hero

Secção inicial da página.

```

<header class="factory-grid overflow-hidden">
  <nav class="mx-auto flex max-w-7xl items-center justify-between px-6 py-6">
    <div class="flex items-center gap-3">
      <div class="grid h-11 w-11 place-items-center rounded-2xl bg-graphite text-concrete shadow-
lg">■</div>
      <div>
        <p class="text-sm font-bold uppercase tracking-[-.28em] text-steel">RoboFlux</p>
        <p class="text-xs text-steel">Robótica • Automação • Algoritmia</p>
      </div>
    </div>
    <a href="#algoritmo" class="rounded-full bg-graphite px-5 py-2 text-sm font-semibold text-concrete
transition hover:bg-steel">Ver algoritmo</a>
  </nav>

```

```

<section class="mx-auto grid max-w-7xl items-center gap-12 px-6 pb-20 pt-10 lg:grid-cols-[1.05fr_.95fr]
  lg:pb-28">
  <div>
    <span class="inline-flex rounded-full border border-steel/20 bg-white/45 px-4 py-2 text-sm font-
semibold text-steel backdrop-blur">Curso de Robótica e Automação Industrial</span>
    <h1 class="mt-7 max-w-4xl text-5xl font-black leading-tight tracking-tight text-graphite
md:text-7xl">
      Sistemas industriais que pensam, medem e decidem.
    </h1>
    <p class="mt-6 max-w-2xl text-lg leading-8 text-steel">
      Sensores recolhem dados, controladores processam sinais e algoritmos escolhem o caminho mais
eficiente para a produção.
    </p>
    <div class="mt-9 flex flex-wrap gap-4">
      <a href="#algoritmo" class="rounded-full bg-sage px-7 py-3 font-bold text-graphite shadow-lg
transition hover:-translate-y-1">Simular rota AGV</a>
      <a href="#conceitos" class="rounded-full border border-graphite/15 bg-white/55 px-7 py-3 font-bold
text-graphite backdrop-blur transition hover:-translate-y-1">Conceitos-chave</a>
    </div>
  </div>

  <div class="relative rounded-[2rem] border border-white/60 bg-white/55 p-6 shadow-panel backdrop-
blur">
    <div class="absolute -right-6 -top-6 h-28 w-28 rounded-full bg-copper/70 blur-2xl"></div>
    
  </div>
</section>
</header>

```

HTML/Astro — conceitos

Cartões explicativos da área de robótica e automação.

```

<section id="conceitos" class="mx-auto max-w-7xl px-6 py-20">
  <div class="grid gap-5 md:grid-cols-3">
    <article class="rounded-3xl bg-white/65 p-7 shadow-lg backdrop-blur">
      <p class="text-3xl">■</p>
      <h2 class="mt-5 text-xl font-black">Sensores</h2>
      <p class="mt-3 text-steel">Leem distância, posição, temperatura e presença de peças na linha.</p>
    </article>
    <article class="rounded-3xl bg-white/65 p-7 shadow-lg backdrop-blur">
      <p class="text-3xl">■</p>
      <h2 class="mt-5 text-xl font-black">Controladores</h2>
      <p class="mt-3 text-steel">PLCs e microcontroladores transformam entradas em decisões seguras.</p>
    </article>
    <article class="rounded-3xl bg-white/65 p-7 shadow-lg backdrop-blur">
      <p class="text-3xl">■</p>
      <h2 class="mt-5 text-xl font-black">Atuadores</h2>
      <p class="mt-3 text-steel">Motores, válvulas e braços robóticos executam o movimento físico.</p>
    </article>
  </div>
</section>

```

HTML/Astro — conclusão

Conclusão e chamada para visitar a página.

```

<section id="conclusao" class="mx-auto max-w-7xl px-6 pb-24">
  <div class="grid gap-6 rounded-[2rem] bg-white/70 p-8 shadow-panel backdrop-blur lg:grid-cols-[1fr_.8fr]
  lg:p-10">
    <div>
      <p class="font-bold uppercase tracking-[.28em] text-steel">Conclusão</p>
      <h2 class="mt-4 text-4xl font-black leading-tight text-graphite md:text-5xl">
        A robótica industrial não é magia: é sensores, decisões e movimento seguro.
      </h2>
      <p class="mt-5 max-w-3xl text-lg leading-8 text-steel">
        Este projeto mostra como um AGV pode receber um mapa, evitar obstáculos e calcular uma rota usando
o algoritmo A*. A página junta programação, algoritmia, design web e automação industrial num exemplo
visual e interativo.
      </p>
    </div>

    <div class="rounded-3xl bg-graphite p-7 text-concrete">
      <h3 class="text-2xl font-black">Experimenta a simulação</h3>
      <p class="mt-4 leading-7 text-concrete/75">
        Gera um novo mapa, executa o algoritmo e observa como o robô decide o caminho passo a passo.
      </p>
      <a href="#algoritmo" class="mt-7 inline-flex rounded-full bg-sage px-6 py-3 font-black text-graphite
transition hover:-translate-y-1 hover:bg-copper">
        Ir para o algoritmo
      </a>
    </div>
  </div>
</section>

```

```
</div>  
</div>  
</section>
```


9. Conclusão

O RoboFlux cumpre o objetivo do projeto final porque transforma conceitos de programação em algo visível e ligado à Robótica e Automação Industrial. A página mostra estrutura HTML, estilos CSS, interatividade com JavaScript, manipulação do DOM, funções, ciclos, condicionais, estruturas de dados e um algoritmo real de planeamento de rota.

Mais importante: o projeto não foi feito só para “ficar bonito”. Foi feito para explicar uma lógica. O mapa muda, o AGV precisa de tomar decisões e o algoritmo mostra passo a passo como chega ao destino. É exatamente isto que interessa numa apresentação: entender o processo, não decorar código.

Página final publicada

<https://roboflux-robotica.pages.dev/>

Repositório GitHub

<https://github.com/JojoDeveloper01/roboflux-robotica>

Bibliografia e referências

- Documentação oficial do Astro — <https://docs.astro.build/>
- Documentação Tailwind CSS — <https://tailwindcss.com/docs>
- Cloudflare Pages — <https://developers.cloudflare.com/pages/>
- Conceito do algoritmo A* aplicado a procura de caminhos em grelhas.